

DRUPAL UPGRADE THE RIGHT WAY

Core & module updates across Acquia Cloud environments — a repeatable, low-risk workflow every site builder needs.



GOLDEN RULE

All code changes happen LOCALLY in a feature branch — never directly on Acquia. Local → Acquia Dev → Stage → Production.

YOUR ACQUIA ENVIRONMENT PIPELINE

ALL WORK STARTS HERE

Your Local Environment **01**

All Composer updates, patches, and code changes happen locally in a feature branch. DDEV, Lando, or your local stack — never on Acquia directly.

INTEGRATION → VALIDATE

Acquia Dev → Stage **02**

Merge your branch into Dev to integrate with other work. Then promote to Stage — a prod mirror — for full regression testing and stakeholder signoff.

FINAL DESTINATION

Acquia Production **03**

Deploy only after Stage is clean and signed off. Take a DB backup first. Run database updates and clear cache immediately after deploy.

STEP-BY-STEP UPGRADE WORKFLOW

LOCAL — PREPARATION

1 Pull Prod DB to Local & Create Branch

Use `accli pull:db` or Acquia Cloud UI to pull the latest prod database into your local environment. Branch from **main/master** before touching anything: `update/core-11.x-YYYY-MM`.

LOCAL — UPDATE

2 Run Targeted Composer Update Locally

Do **NOT** run `composer update` with no args — updates everything at once, rollback nightmare. Target **core first**, then contrib modules in logical groups (e.g., all Media-related together).

LOCAL — UPDATE

3 Run DB Updates & Test Locally

Run `drush updb -y`, then `drush cim -y` if config changed, then `drush cr`. Test all content types, Views, Layout Builder, and key user flows locally before touching Acquia.

LOCAL — COMMIT

4 Commit & Push Your Feature Branch

Commit `composer.json`, `composer.lock`, and any exported config changes. Push your branch. Open a PR for team review — no direct merges to main without review.

ACQUIA DEV — INTEGRATION

5 Merge to Dev & Deploy to Acquia Dev

After PR approval, merge your branch into the **dev integration branch** and deploy to Acquia Dev. Run `drush updb` and `drush cim`. Confirm the build is stable and no conflicts with other recent work.

ACQUIA STAGE — PROMOTE

6 Sync Prod DB to Stage & Deploy

Copy the **prod DB fresh to Stage** via Acquia Cloud immediately before testing. Deploy the same code. Run `drush updb` and `drush cim`. This is your production simulation — take it seriously.

ACQUIA STAGE — VALIDATION

7 Stakeholder Signoff on Stage

Have a content editor or site owner walk through critical paths. Check Acquia Insight. Confirm **automated tests pass**. Get explicit signoff before any prod deploy.

ACQUIA PROD — DEPLOY

8 Backup, Deploy to Prod & Run Updates

Deploy during a **low-traffic window**. Take a manual DB backup first. Enable maintenance mode → deploy code → `drush updb` → `drush cim` → `drush cr` → disable maintenance mode.

POST-DEPLOY — MONITOR

9 Monitor Production & Document

Watch Acquia logs and New Relic for **30–60 minutes** post-deploy. Monitor `/admin/reports/dblog` for new errors. Merge dev integration branch back to main if needed. Document what was updated, the date, and any issues in your changelog.

KEY COMPOSER & DRUSH COMMANDS

UPDATE CORE ONLY

```
# Core + core-recommended only
composer update drupal/core \
  drupal/core-recommended \
  drupal/core-composer-scaffold \
  --with-all-dependencies
```

UPDATE CONTRIB MODULE GROUP

```
# Target related modules together
composer update drupal/views \
  drupal/views_infinite_scroll \
  drupal/views_bulk_operations \
  --with-all-dependencies
```

DRY RUN — SEE WHAT CHANGES

```
# Preview before committing
composer update --dry-run \
  drupal/core-recommended

# Check all contrib:
composer outdated "drupal/*"
```

POST-UPDATE DRUSH SEQUENCE

```
# Always run in this exact order
drush updb -y
drush cim -y
drush cr

# Export config after cim on prod:
drush cex -y && git add . && git commit
```

CRITICAL RULES & COMMON MISTAKES



Never Composer Update All

Running `composer update` with no arguments updates every dependency at once. Impossible to isolate what broke when something goes wrong.



Never Skip the DB Backup

Always snapshot prod via Acquia Cloud before deploying. Acquia keeps automated backups, but a manual one immediately before updates is non-negotiable.



Never Edit Files on Acquia Directly

Acquia environments reset on deploy. All changes must flow through Git. FTP/SSH file edits on prod get wiped and create dangerous out-of-sync states.



Group Related Module Updates

Update modules with shared dependencies together (e.g., all Media ecosystem modules). Partial updates cause version constraint conflicts in Composer.



Run drush updb After Every Deploy

Forgetting database updates after a code deployment causes subtle data corruption and cryptic errors. Put it in your deploy checklist every single time.



Use Acquia CLI for All Env Ops

The `accli` tool lets you copy databases, deploy branches, and run drush commands across environments without leaving your terminal.

[DrupalHelps.com](https://drupalhelps.com)

Expert Drupal Training for Site Builders — Drupal 10 & 11

TEST ON DEV.
VALIDATE ON STAGE.
DEPLOY TO PROD.